

Statistics 2A Coursework

Candidate number: 23271

Question 1:

a)

When constructing this confidence interval for μ we must account for the fact that σ^2 is unknown, and thus we must approximate it using S^2 , meaning we must use the t-distribution with $n - 1$ degrees of freedom (n being the number of observations) to calculate the upper and lower bounds for the confidence interval.

```
meanCI = function(datavec,alpha){
  sd = sd(datavec)
  mu = mean(datavec)
  n = length(datavec)
  lo = mu - qt(1-alpha/2,n-1)*(sd/sqrt(n))
  hi = mu + qt(1-alpha/2,n-1)*(sd/sqrt(n))
  return(c(lo, hi))
}
```

b)

Similarly when constructing the confidence interval for σ^2 we must account for the fact that μ is unknown, and thus we use the chi-squared distribution (with $n - 1$ degrees of freedom) to calculate the upper and lower bounds.

```
varCI = function(datavec,alpha){
  var = var(datavec)
  n = length(datavec)
  lo = ((n-1)*var)/qchisq(1-alpha/2,n-1)
  hi = ((n-1)*var)/qchisq(alpha/2,n-1)
  return(c(lo,hi))
}
```

Question 2:

a)

First we load the data using the provided link

```
newcomb = scan("http://people.bath.ac.uk/masss/ma20226/LabSheets/newcomb.txt")
```

We can now calculate the 92.5% confidence interval for μ

```
meanCI(newcomb,0.075)
```

```
## [1] 14.32512 29.17488
```

As well as the 94% confidence interval for σ^2

```
varCI(newcomb,0.06)
```

```
## [1] 183.5873 641.6812
```

The first thing that stands out to me is the large width of the interval for variance, which suggests that the data is not normally distributed (due to the empirical rule for standard deviation).

b)

We can eliminate the two smallest values using the sort function as follows.

```
newcomb_top18 = sort(newcomb)[3:20]
```

We then calculate the new confidence intervals in the same way.

```
meanCI(newcomb_top18,0.075)
```

```
## [1] 24.12999 29.31446
```

```
varCI(newcomb_top18,0.06)
```

```
## [1] 19.36176 72.98025
```

In the confidence interval for μ we see a shrink in its size coming from a large increase in the lower bound; which makes sense as the values eliminated were the smallest, meaning on average the data values are greater.

Similarly we see a shrink in the size of the σ^2 confidence interval, however this change is much more drastic (change in width from ~ 458 to ~ 53.6), understandably so as now that the two smallest values are gone (in particular -44) the spread of the data is much smaller, and as such there is the same level of confidence that the variance is contained in a much more concentrated range.

From observing these new confidence intervals it can be seen as more likely that the data has come from a normal distribution.

Question 3

```
ciplot <- function(confint, true)
{
  nInt <- length(confint[, 1])
  ymat <- matrix(c(1:nInt,1:nInt),nrow=nInt,ncol=2)
  yvec <- c(ymat)
  xvec <- c(confint)
  plot(xvec, yvec, type="n", ylab="Sample number",
  main=expression("Plot of realised CIs"), xlab="Parameter")
  abline(v=true)
  for (i in 1:nInt)
  {
    if (xvec[i] <= true & xvec[i+nInt] >= true)
    {
      lines(c(xvec[i],xvec[i+nInt]),c(yvec[i], yvec[i+nInt]),col=2)
    }
    else
    {
      lines(c(xvec[i],xvec[i+nInt]),c(yvec[i], yvec[i+nInt]),col=4)
    }
  }
}
```

The function first defines the following variables, `nInt` is 'm', the number of realised intervals given in the argument 'confint', `ymat` is an $m \times 2$ matrix where the elements of each row are the column numbers (for both columns), `yvec` then translates this matrix to a vector, in which lists the elements of the first column, and then the second. Finally `xvec` is a vector containing all the confidence interval bounds, listing all the lower bounds, and then all the corresponding upper bounds afterwards (both in the 1-m order given in `confint`).

The function then creates an empty graph (by using `type='n'`), using `xvec` and `yvec` as the axis values (and adding some relevant labels), as well as a vertical line at `x=true`, the true parameter value. The 'if' statement then determines whether or not the true parameter value is contained within each confidence interval; a red line (`col=2`) is plotted for the interval if it contains the true value, and a blue line (`col=4`) if not.

Question 4

In order to carry out the simulation, I first created a matrix of samples generated from the given normal distribution ($N(\mu = 5, \sigma^2 = 4)$). I then created an empty matrix to then store the confidence interval bounds in using a for-loop.

```
set.seed(1)
samples = matrix(rnorm(2000, mean=5, sd=2), nrow=100, ncol=20)

mean_intervals = matrix(0, nrow=100, ncol=2)

for(i in 1:100){
  interval_i = meanCI(samples[i,],alpha=0.1)
  mean_intervals[i,1] = interval_i[1]
  mean_intervals[i,2] = interval_i[2]
}
```

Using the 'ciplot' function we can then create a plot of these confidence intervals.

```
ciplot(mean_intervals, true=5)
```



The plot shows us that out of the 100 simulations, 10 intervals did not contain the parameter (number of blue lines), meaning we have 90% coverage for μ which aligns well (perfectly for this seed) with our value of alpha (corresponding to 90% confidence).

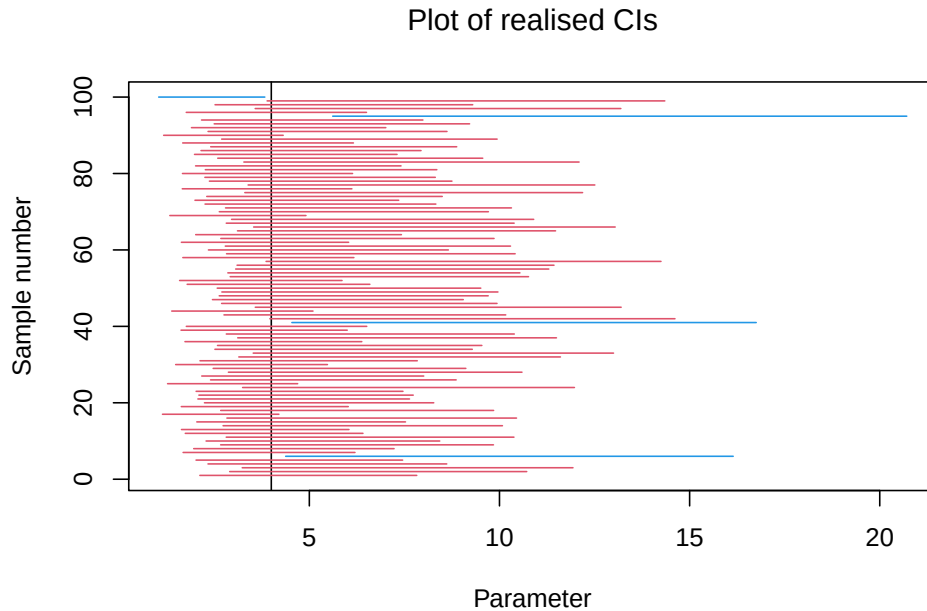
Question 5

Using the same matrix of samples we can create a plot of realised confidence intervals for σ^2

```
var_intervals = matrix(0, nrow=100, ncol=2)

for(i in 1:100){
  interval_i = varCI(samples[i,], alpha=0.05)
  var_intervals[i,1] = interval_i[1]
  var_intervals[i,2] = interval_i[2]
}

ciplot(var_intervals, true=4)
```



For this plot we should expect to see fewer blue lines (than in the previous plot) as we are using a higher confidence level. Thus we observe that out of the 100 simulations, 4 intervals did not contain the parameter, giving 96% coverage which is very close to the confidence level of 95%, and a higher coverage than for the previous plot (as expected).

By comparing this plot to the one previous we see that the true value lies much more to the left of the plot (meaning the intervals are not very symmetric about the true value); which shows us the difference between using the symmetric t-distribution and the non-symmetric chi-square distribution for evaluating confidence intervals for the respective parameters μ and σ^2

Question 6

S^2 is an unbiased estimator, and as such $MSE(S^2) = Var(S^2)$ which we can calculate using simulation as follows:

```
MSE_S = function(n, nSim, v){
  sim_var = array(0, nSim)
  for(i in 1:nSim){
    sim = rt(n, df=v)
    sim_var[i] = var(sim)
  }
  return(var(sim_var))
}
```

This function allows us to calculate to the mean squared error of S^2 for a singular sample size and number of simulations, for example:

```
MSE_S(100, 1000, 5)
```

```
## [1] 0.1581562
```

We can now use a for-loop to calculate the MSE for various values of n , meaning we can also determine the relationship between n (size of each simulation) and MSE.

First I created an empty array to then store the MSE values in for each respective n value, (I felt $n_{\text{Sim}}=1000$ was appropriate as it is enough simulations to yield accurate results without causing long/costly run-times).

```
set.seed(100)
MSEs = rep(0,6)
n_values = c(10,100,500,1000,5000,10000)
for(i in 1:6){
  MSEs[i] = MSE_S(n_values[i], 1000, 5)
}
MSEs
```

```
## [1] 1.852679076 0.178058540 0.035839735 0.020933496 0.003949297 0.002795686
```

From observing the mean-squared error at increasing values of n we see that it starts off rather high (1.85 for $n = 10$) but then rapidly decreases and gets smaller with each increases in n . This shows that although we cannot compute infinite simulations, the trend dictates that the mean squared error of S^2 approaches 0 as n gets large; and as such we can conclude that S^2 is a consistent estimator of σ^2 . From this we can deduce that as n gets larger, S^2 will become arbitrarily close to the true σ^2 .

Question 7

To create this plot we first create sequences to step-up the n and ν values of which to produce the plot points at.

```
vseq = seq(5,25,5)
nseq = seq(15,195,10)
```

Using these sequences we can then create a plot of the mean squared errors for S^2 at different ν values with increasing n . To achieve this we use the previously created MSE_S function to determine the MSE for each ν value across the increasing sequence of n values (using a nested for-loop).

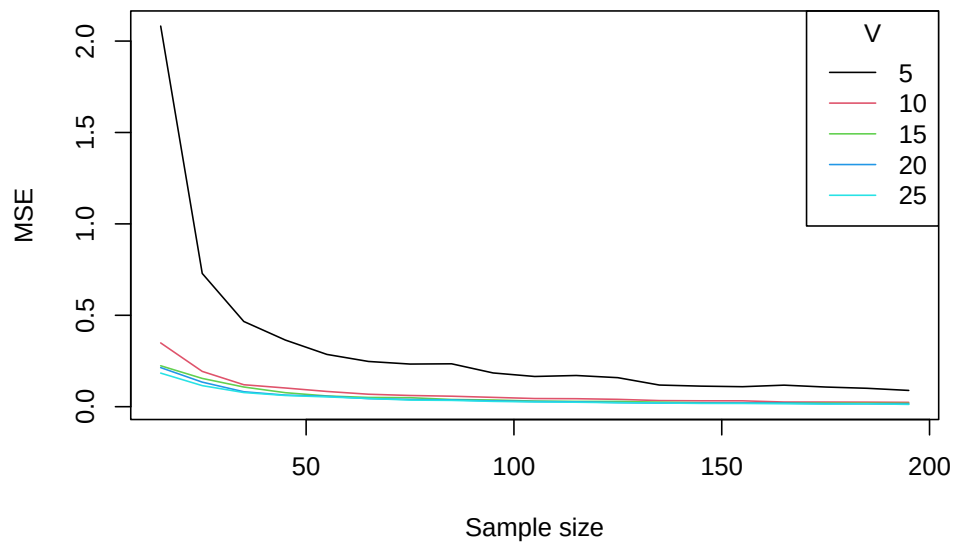
```
MSEarray = array(0, dim=c(length(nseq), length(vseq)))

set.seed(13)
for(i in 1:length(nseq)){
  for(j in 1:length(vseq)){
    MSEarray[i,j] = MSE_S(n=nseq[i], nSim=1000, v=vseq[j])
  }
}

plot(type='l', nseq, MSEarray[,1], xlab="Sample size", ylab="MSE",
     main="Behaviour of MSE for increasing v and n", ylim=c(min(MSEarray), max(MSEarray)))

for(j in 2:length(vseq)){
  lines(type='l', nseq, MSEarray[,j], col=j)
}
legend('topright', col=1:length(vseq), legend=vseq, lty=1, title="V")
```

Behaviour of MSE for increasing v and n



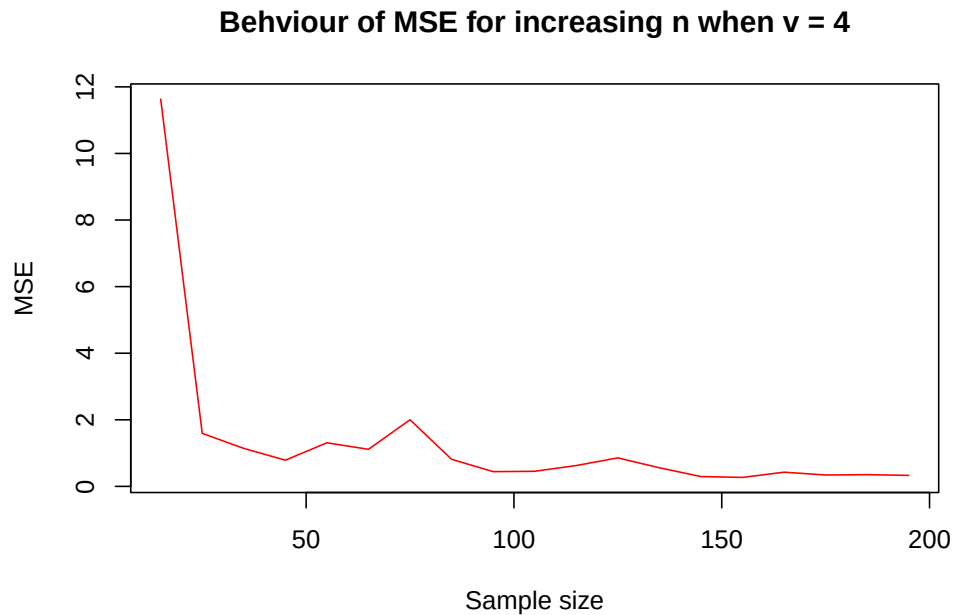
The plot shows us what we saw previously for $\nu = 5$ in that the MSE steeply decreases before approaching 0 for increasing n , it also shows us that for increasing degrees of freedom ν , this drop-off happens much more quickly, shown by the lines on the graph starting already much closer to 0; indicating that with fewer degrees of freedom there is greater variation, particularly for smaller sample sizes.

Question 8

Similarly to simulating the MSEs in the previous question we do so for specifically $\nu = 4$, using the same sequence of n and number of simulations so results can be compared fairly.

```
MSE_4 = rep(0, length(nseq))

set.seed(3)
for(i in 1:length(nseq)){
  MSE_4[i] = MSE_S(n=nseq[i], nSim=1000, v=4)
}
plot(type='l', nseq, MSE_4, xlab="Sample size", ylab="MSE",
     main="Behaviour of MSE for increasing n when v = 4", col='red')
```



We see again that when degrees of freedom is low there is room for greater variation in the data (as there is a higher probability of extreme values), hence the graph does not smoothly decrease, similar to the black line representing $\nu = 5$ in the previous plot.

We do however see that the general behaviour of the curve reflects what is seen previously, in that we see a steep decrease in MSE and then a gentle approach to 0 as n increases.

Question 9

To perform this simulation study for various degrees of freedom we first need to create a function to calculate the coverage when simulating confidence intervals for a set value of ν . To create this function I decided to use the previously defined 'varCI' function to calculate the many interval bounds, and then return the proportion of these intervals that contain the true σ^2 value.

As we are performing a study for changing ν , it is important that we keep n and $nSim$ constant for simulations (I have decided to use 100 and 1000 respectively as it seems enough to yield reliable results without being too costly).

```
coverage = function(n, nSim, v){
  true_var = v/(v-2)
  CIarray = array(0, dim=c(nSim,2))
  for(i in 1:nSim){
    CIarray[i,] = varCI(rt(100,v),0.05)
  }
  return(mean((CIarray[,1]<true_var) * (true_var<CIarray[,2])))
}
```

We can now use this function to determine the coverage for a singular ν value, for example:

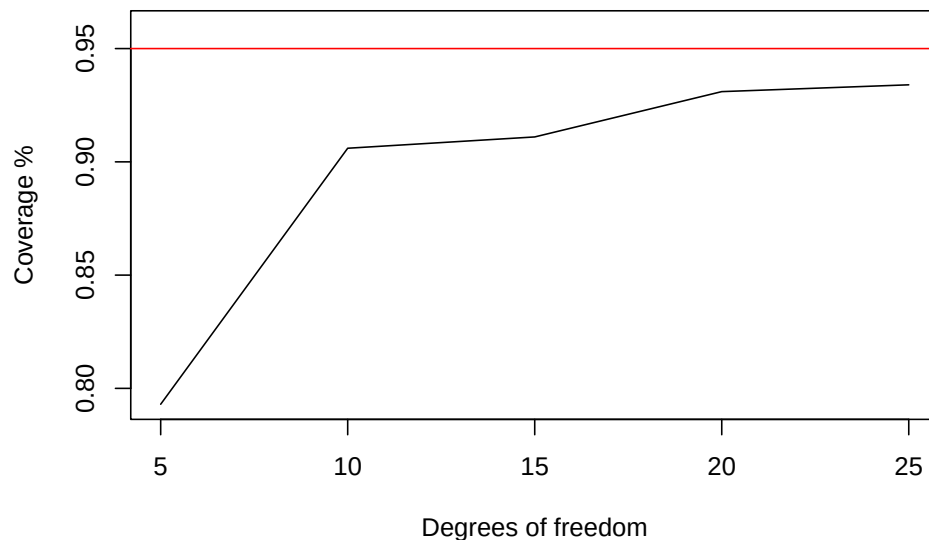
```
coverage(100,1000,5)
```

```
## [1] 0.772
```

This value is far below the confidence level of 95%, this is our first indication (from the results) that the model is not robust to model misspecification, as if the model were robust then the coverage would well-reflect the confidence level (i.e they would be very similar).

We can further assess this robustness across the range $\nu \in (5, 25)$ using a for-loop.

```
set.seed(2)
coverages = rep(0, length(vseq))
for(i in 1:length(vseq)){
  coverages[i] = coverage(100,1000,vseq[i])
}
plot(vseq,coverages, type='l', ylim=c(coverages[1],0.96),
     ylab="Coverage %", xlab="Degrees of freedom")
abline(h=0.95, col='red')
```

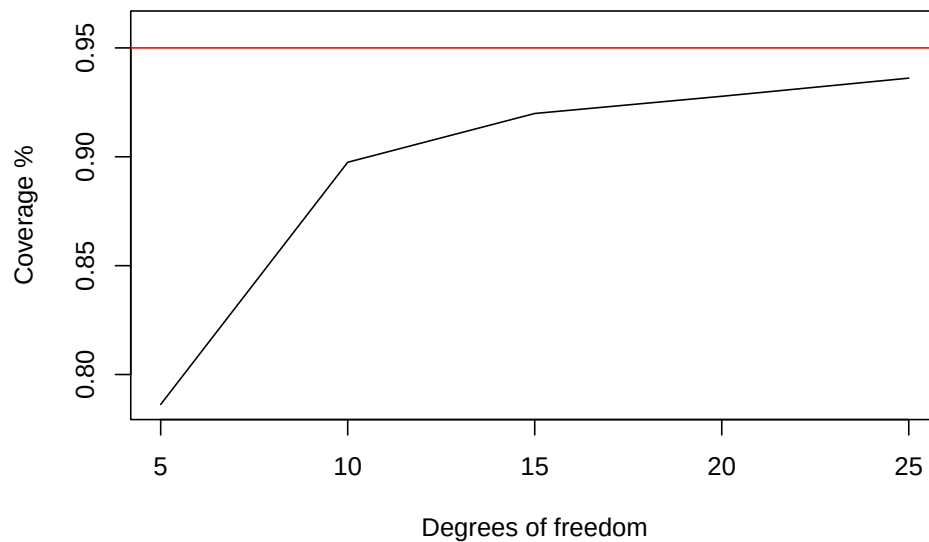


The plot shows that as ν increases, we also see an increase in the coverage percentage; however we do not see the coverage ever meet the expected coverage of 95% (indicated by the red line), which installs confidence in the claim that the model is **not** robust to model misspecification. Although our simulated coverage does not reach the expected level of 95%, we do see the line (on the plot) approach the 95% line as ν increases, which shows the asymptotic approach of the t-distribution to the normal distribution. However as we are assessing the distribution for $\nu \in (5, 25)$ we cannot draw conclusions on robustness from the distribution's behaviour outside this range.

To further verify this claim we can run a greater amount of simulations (not done originally to save run-time), as a low number of simulations has the potential to yield incorrect conclusions due to the random nature

of the data. Running this simulation will help to determine the reason for the lower than expected level of coverage, to aid in verification of our robustness assessment.

```
set.seed(2)
coverages = rep(0, length(vseq))
for(i in 1:length(vseq)){
  coverages[i] = coverage(100,100000,vseq[i])
}
plot(vseq,coverages, type='l', ylim=c(coverages[1],0.96),
     ylab="Coverage %", xlab="Degrees of freedom")
abline(h=0.95, col='red')
```



Here (for $nSim = 100000$) we see a very similar graph, which shows validity in our robustness assessment independent of the number of simulations ran; which allows me to confidently conclude that the model is not robust to model misspecification.

Question 10

Using the results from the previous question we can see that a lower than expected number of intervals successfully cover the parameter. Lengthening the intervals (by increasing the confidence level) would mean there is a greater chance that the true parameter is contained within/covered by them, this therefore increases the coverage. A longer/wider interval would however decrease the precision of our estimate as there is a greater level of uncertainty surrounding the true value.

When we use an interval to estimate the variance we do so by inverting a pivot to create a range surrounding a point estimate for the true parameter, with the interval width corresponding (inversely) to how precise the estimate is, thus a wider interval corresponds to a less precise estimate.